

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий
Дисциплина «Естественно-языковой интерфейс интеллектуальных систем»

ОТЧЁТ
к лабораторной работе №1
на тему
**«РАЗРАБОТКА АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ
ФОРМИРОВАНИЯ СЛОВАРЯ ЕСТЕСТВЕННОГО ЯЗЫКА»**

Выполнил студент группы 321701
СЕМЕНЯКО Владимир Дмитриевич

(дата, подпись студента)

Проверил
КРАПИВИН Юрий Борисович

(дата, подпись преподавателя)

Минск 2026

1 ЗАДАНИЕ

Спроектировать и программно реализовать структуры хранения данных, алгоритмы их обработки, необходимые в рамках базовых требований к разрабатываемому приложению

Вариант 21

- а) Язык текста - *английский*
- б) Формат входного документа - *DOC, DOCX*
- в) Вариант задания - *Вариант 3*.

Описание варианта: Список слов, упорядоченный по алфавиту и включающий только лексемы с дополнительно оформленными записями для образования словосочетаний. В этих записях должны храниться слова (точнее, ссылки на эти слова), с которыми данное слово может сочетаться. При этом возможно обеспечение автоматизированного извлечения из исходных текстов типовых словосочетаний с их последующей обработкой. *Можно добавить средства синтеза словосочетаний, но при этом в словарь необходимо будет добавить правила формирования словоформ (см. задание 2).

Задачи:

- а) Познакомиться с назначением, структурой и функциональностью, предоставляемой базовым ЛП для решения задачи автоматического лексического и лексико-грамматического анализа ТЕЯ.
- б) Закрепить навыки программирования при решении задач автоматической обработки ТЕЯ.

2 ОПИСАНИЕ СТРУКТУР ХРАНЕНИЯ ДАННЫХ И АЛГОРИТМОВ ОБРАБОТКИ

2.1 Структуры данных

– **Текст документа** — строка (`str`), результат чтения или извлечения из файла. Кодировка UTF-8.

– **Список лемм** — список строк `list[str]`, порядок соответствует порядку слов в тексте. Получается после токенизации и лемматизации (NLTK: `word_tokenize`, `pos_tag`, `WordNetLemmatizer`).

– **Словарь лексем (dictionary)** — словарь вида `dict[str, list[dict]]`, где ключ — лемма (строка), значение — список записей `{"word": str, "freq": int}` (лексема-партнёр и частота совместной встречи в биграмах). Леммы упорядочены по алфавиту при выводе; для каждой леммы парт-

нёрты отсортированы по убыванию частоты.

- **Типовые словосочетания (full_collocations)** — список пар `list[tuple[str, int]]`: элемент — ((слово1, слово2), частота).

- **Примечания к леммам (lemma_notes)** — словарь `dict[str, str]`: лемма → произвольная текстовая запись. Сохраняется и загружается вместе со словарём в JSON.

- **Сохранённый словарь (JSON)** — объект с ключами "lemmas", "notes", "collocations" (список объектов с полями w1, w2, freq). Обеспечивает сохранение, загрузку и перенос словаря между сессиями.

2.2 Структурно-функциональная схема разработанного приложения

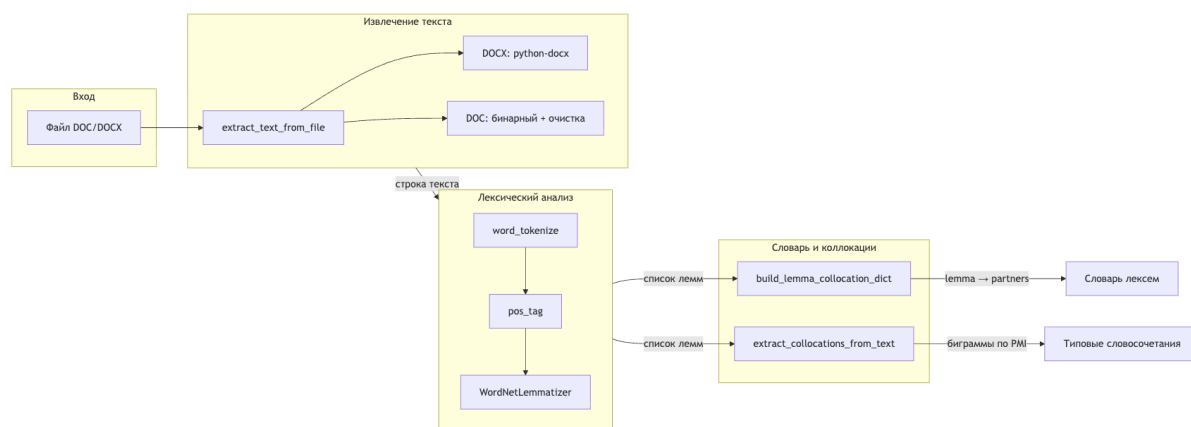


Рисунок 1 – Общий поток обработки

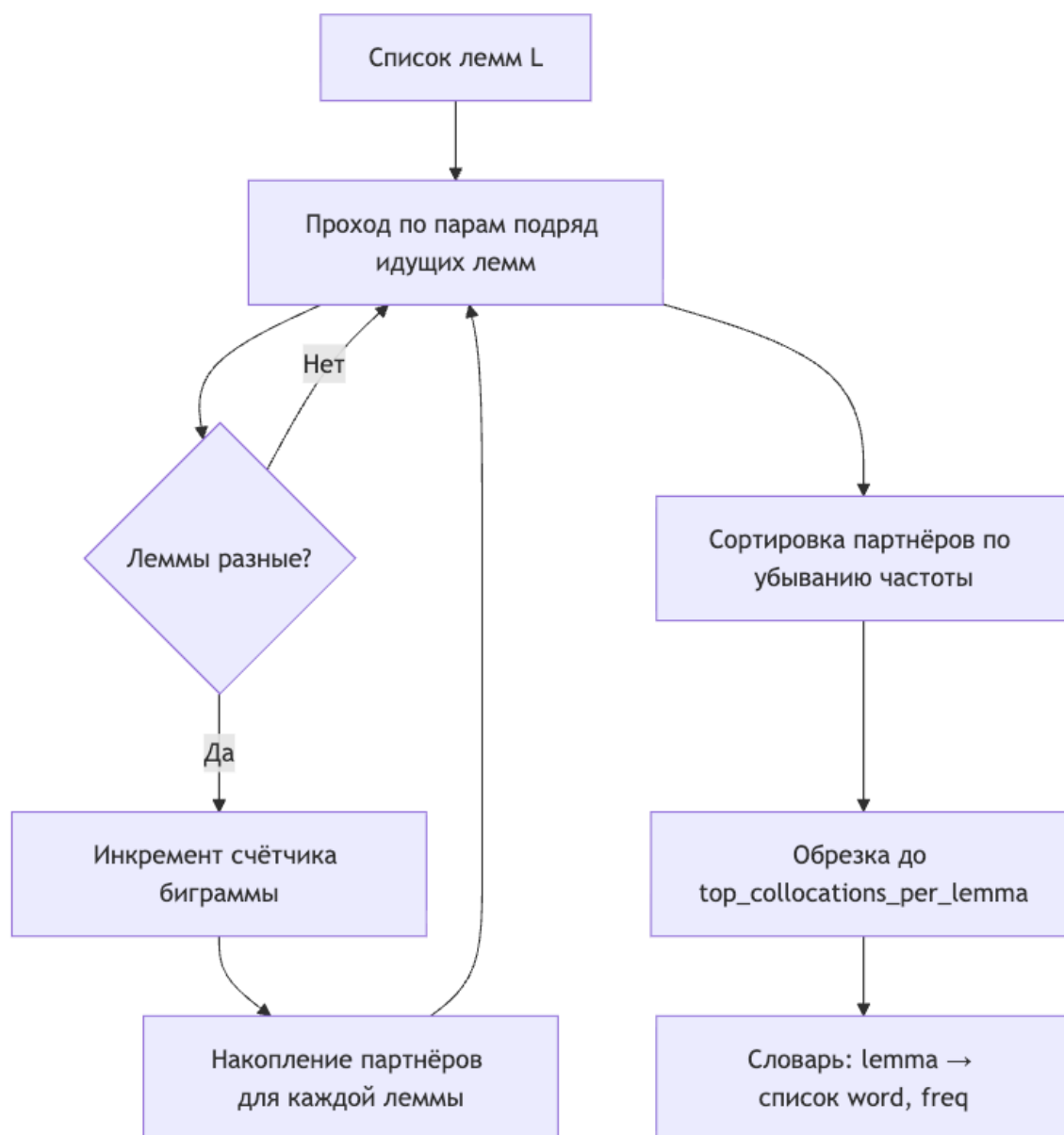


Рисунок 2 – Алгоритм построения словаря сочетаемости

2.3 Алгоритмы обработки (базовые требования)

- **Ввод текста:** по расширению файла вызывается соответствующая ветка: DOCX — python-docx (обход параграфов); DOC — чтение бинарного файла и упрощённая очистка.
- **Лексический анализ:** токенизация (NLTK word_tokenize), отбор буквенных токенов, разметка частей речи (pos_tag), приведение к леммам (WordNetLemmatizer). Результат — последовательность лемм.
- **Построение словаря сочетаемости:** однократный проход по списку

лемм; для каждой пары подряд идущих лемм (a, b) при $a \neq b$ инкрементируется счётчик биграммы и накапливаются «партнёры» с частотами; затем для каждой леммы партнёры сортируются по убыванию частоты и обрезаются до заданного лимита. Сложность $O(L)$, где L — длина списка лемм.

– **Извлечение типичных словосочетаний:** `BigramCollocationFinder` строит частоты биграмм и слов; `apply_freq_filter(min_freq)` отбрасывает редкие биграммы; `nbest(BigramAssocMeasures.pmi, top_n)` вычисляет PMI и возвращает `top_n` биграмм с наибольшим PMI.

– **Поиск и фильтрация:** при отображении таблицы для каждой леммы проверяется совпадение начала с поисковым запросом и условие сумма частот партнёров $\geq$ заданной минимальной частоты. Таблица перестраивается при изменении поиска или порога.

– **Сохранение и загрузка словаря:** сериализация и десериализация структур в JSON (UTF-8); при загрузке — восстановление структур и обновление таблиц GUI.

– **Экспорт в TXT:** последовательная запись для каждой леммы в алфавитном порядке — заголовок леммы, примечание (если есть), список партнёров с частотами. Кодировка UTF-8.

3 ОЦЕНКА БЫСТРОДЕЙСТВИЯ ПРИЛОЖЕНИЯ

Таблица 3.1 – Оценка быстродействия

Номер теста	Количество символов	Время
1	11915	3.111с
2	1914	1.073с
3	1336	1.022

ВЫВОД

– Разработано приложение для автоматического формирования словаря естественного языка (вариант 21: английский язык, форматы DOC/DOCX, задание 3 — словарь лексем с сочетаемостью и извлечение типовых словосочетаний).

– Реализованы требования: входные данные — текст в форматах TXT, RTF, PDF, DOC, DOCX; выходные данные — перечень лексем по алфавиту

с информацией о сочетаемости и список типовых словосочетаний; графический интерфейс; справка; построение, сохранение, загрузка, просмотр, редактирование (примечания), фильтрация и поиск по словарю, экспорт в TXT.

– Использованы библиотеки Python: tkinter (GUI), python-docx (DOCX), NLTK (токенизация, лемматизация, тегирование, биграммы, PMI). Быстродействие приемлемо для документов учебного и среднего объёма.